

+

Informatik 2:

Objektorientiertes Programmieren (OOP) - Teil 2

K

A



©image: various

Heute

- Wiederholung OOP
- Kommunikation
- Vererbung
- Polymorphie

C++

- protected
- virtual
- this
- static

Outlook – Today & Next Week



Heute

- Wiederholung OOP
- Kommunikation
- Vererbung
- Polymorphie

C++

- protected
- virtual
- this
- static

Was ist OOP?



1. Everything is an object
2. Objects communicate by sending and receiving messages
3. Objects have their own memory
4. Every object is an instance of a class
5. The class holds the shared behavior for its instances
6. Program control is passed to the object



„1. Alles ist ein Objekt, 2. Objekte kommunizieren durch das Senden und Empfangen von Nachrichten (welche aus Objekten bestehen), 3. Objekte haben ihren eigenen Speicher (strukturiert als Objekte), 4. Jedes Objekt ist die Instanz einer Klasse (welche ein Objekt sein muss), 5. Die Klasse beinhaltet das Verhalten aller ihrer Instanzen (in der Form von Objekten in einer Programmliste), 6. Um eine Programmliste auszuführen, wird die Ausführungskontrolle dem ersten Objekt gegeben und das Verbleibende als dessen Nachricht behandelt“

– Alan Kay

(Erfinder von Smalltalk & des Begriffs „object oriented“): The Early History of Smalltalk (1993)

https://de.wikipedia.org/wiki/Objektorientierte_Programmierung

Was sind Klassen?



Allgemein:

Jedes Objekt im System kann als ein **abstraktes Modell** eines Akteurs betrachtet werden,

- der Aufträge erledigen,
- seinen Zustand berichten und ändern
- und mit den anderen Objekten im System kommunizieren kann
- ohne offenlegen zu müssen, wie diese Fähigkeiten implementiert.



Was sind Klassen?



Allgemein:

Jedes Objekt im System kann als ein **abstraktes Modell** eines Akteurs betrachtet werden,

- der Aufträge erledigen,
- seinen Zustand berichten und ändern
- und mit den anderen Objekten im System kommunizieren kann
- ohne offenlegen zu müssen, wie diese Fähigkeiten implementiert.



Solche Abstraktionen sind Klassen in der klassenbasierten Objektorientierung.

Begriffsdefinition - Klasse

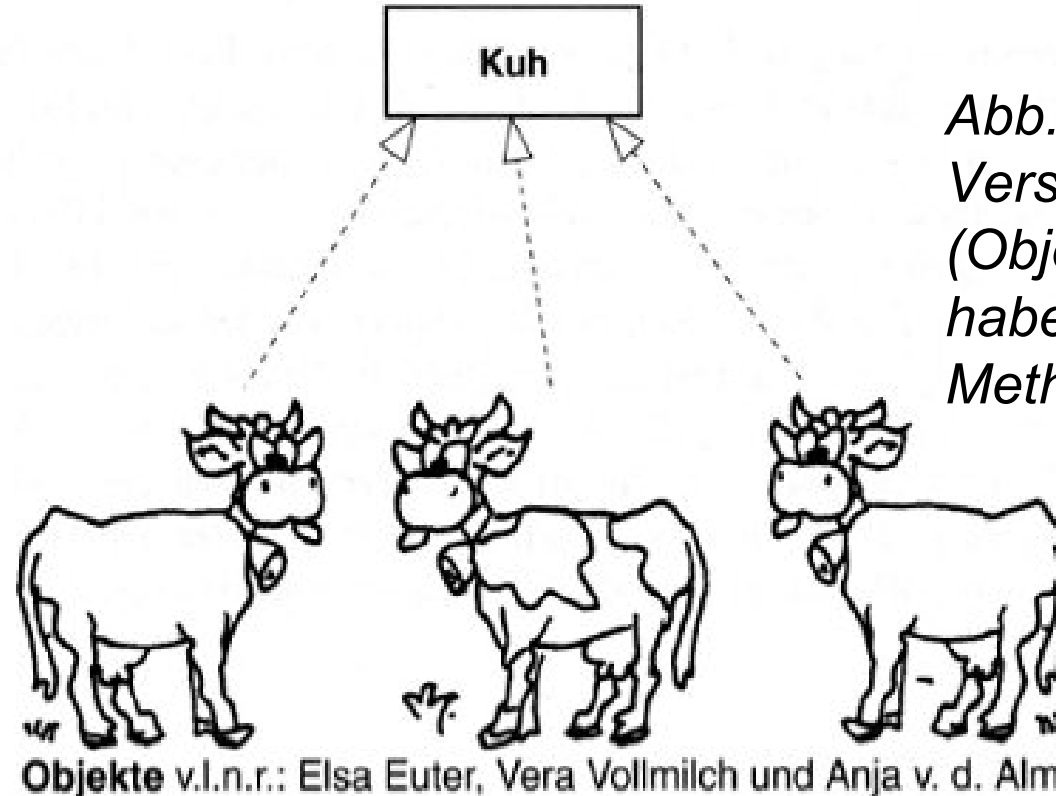


Definition (bitte merken!):

Eine **Klasse** definiert die **Attribute und Methoden** einer Menge **gleichartiger Instanzen/Objekte**. Sie ist ein **Bauplan** für Objekte (im Speicher gibt es sie nicht).

Attribute sind die **Daten** (Variablen) einer Klasse.

Methoden sind die **Funktionen** einer Klasse



*Abb.:
Verschiedene Instanzen
(Objekte) der Klasse „Kuh“
haben die selben Attribute und
Methoden.*

Quelle: Oesterreich (1998)

- **Klassen** sind strukturierte Datentypen, die **Attribute**/Eigenschaften (Variablen) und **Methoden** (Funktionen) zusammenfassen.
- Die Instanzen eines solchen Datentyps werden **Objekte** (oder auch Instanzen) genannt.
- Die einzelnen Attribute und Methoden werden **Mitglied** (engl. **Member**) der Klasse genannt.
- Die Mitglieder einer Klassen können unterschiedliche **Sichtbarkeiten** haben:
 - öffentlich (engl. **public**) – Zugriff von außen (andere Klassen, Hauptprogramm) möglich
 - geschützt (engl. **protected**) – Zugriff nur innerhalb der eigenen Klassenhierarchie
 - privat (engl. **private**) – Zugriff nur aus der eigenen Klasse

Praktische Vorteile von OOP



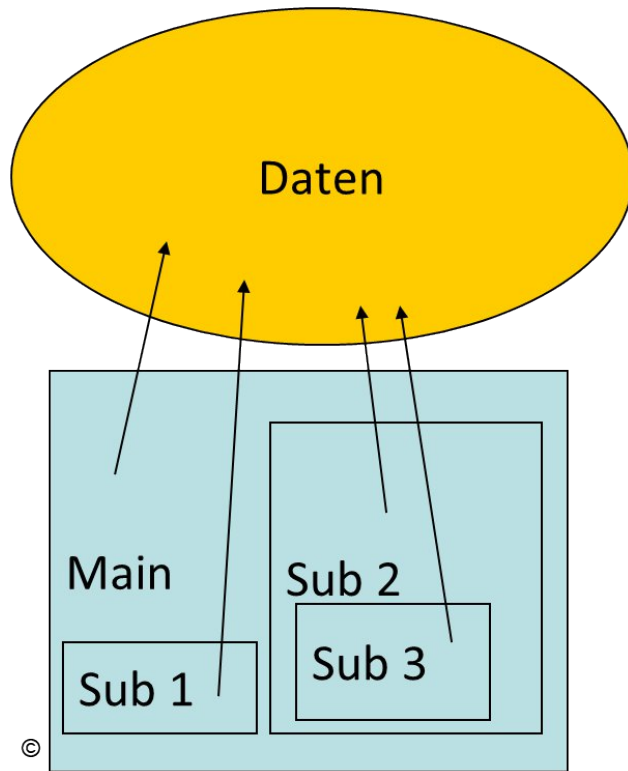
- Spätere Anforderungen können noch leicht nachträglich integriert werden.
- Anspruchsvollere Aufgaben lösbar.
- Bessere **Erweiterbarkeit**
- Bessere **Wiederverwendbarkeit**
- Leichtere Dokumentation
- Näher an der **menschlichen Denkweise**



Vergleich prozedurale Programmierung – OOP

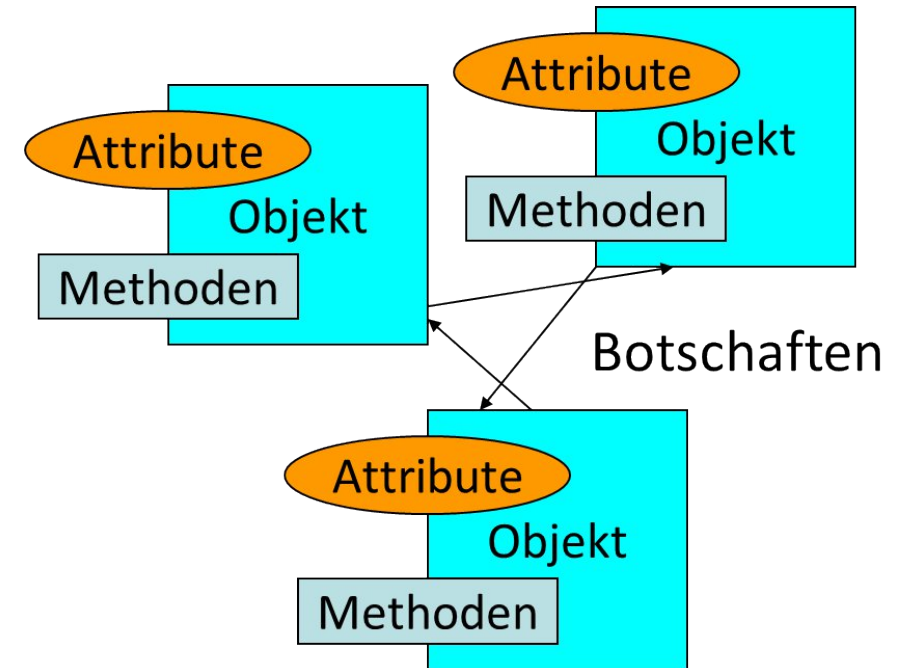


Daten und Funktionen getrennt.
Hauptaugenmerk: Algorithmus.



prozedurale Programmierung

Daten und Funktionen bilden Einheit
(Attribute und Methoden: gekapselt).
Hauptaugenmerk: Kommunikation über Botschaften.

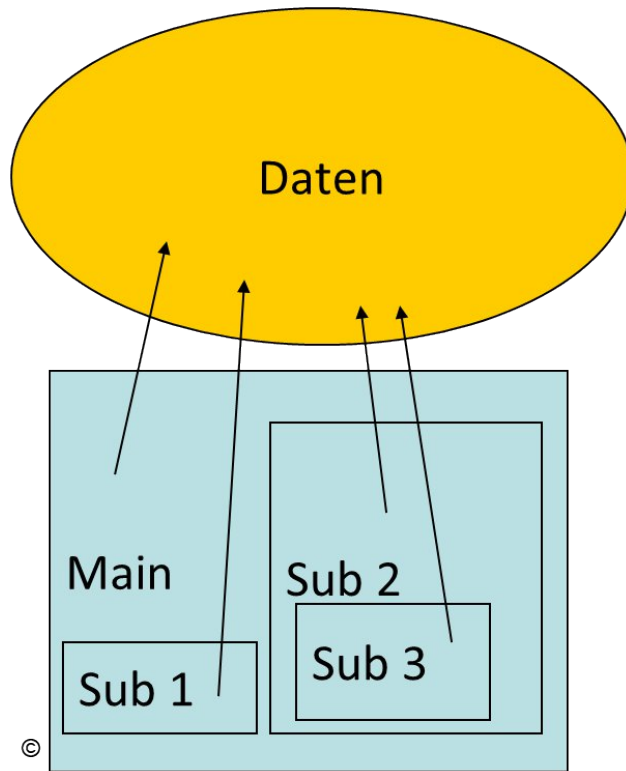


OOP

Vergleich prozedurale Programmierung – OOP



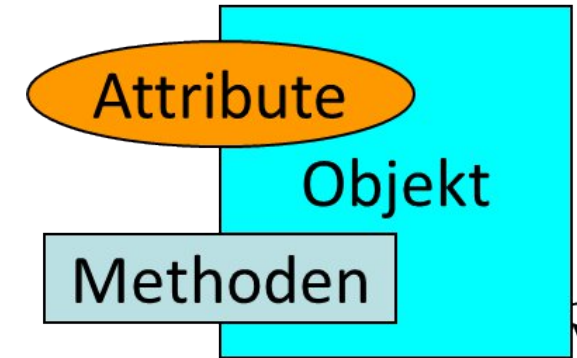
Daten und Funktionen getrennt.
Hauptaugenmerk: Algorithmus.



prozedurale Programmierung

Daten und Funktionen bilden Einheit
(Attribute und Methoden: gekapselt).

Bisher keine Kommunikation zwischen
Objekten angesehen!

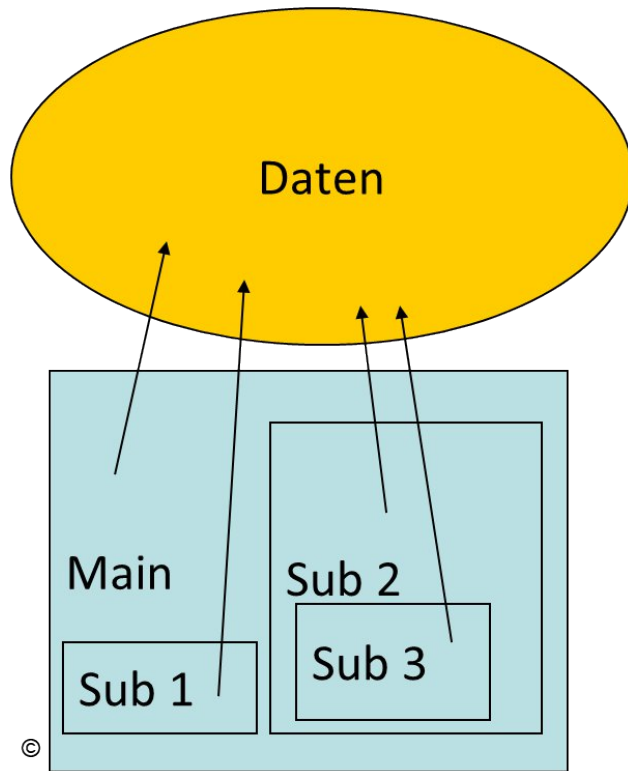


OOP

Vergleich prozedurale Programmierung – OOP

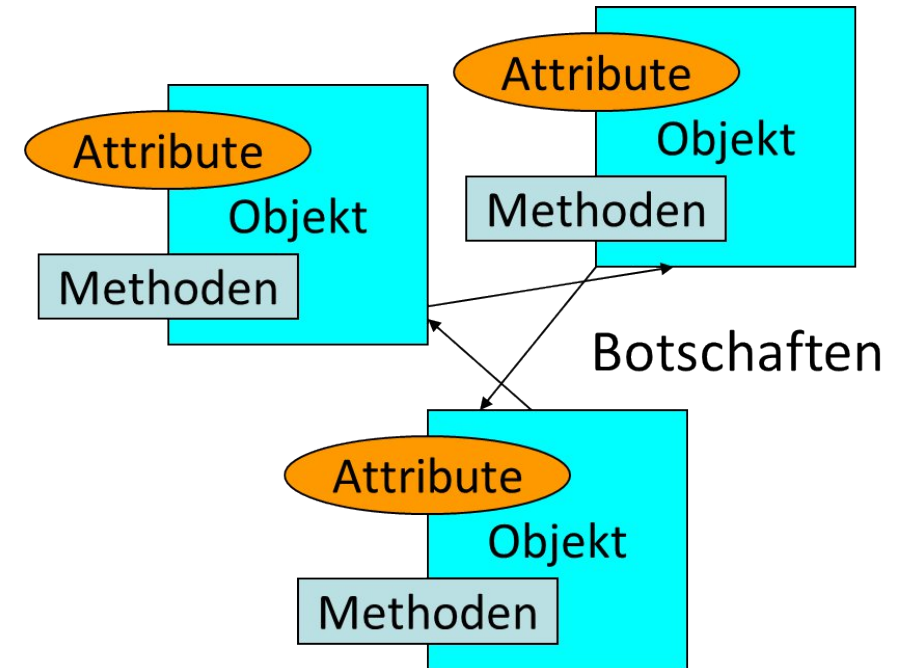


Daten und Funktionen getrennt.
Hauptaugenmerk: Algorithmus.



prozedurale Programmierung

Daten und Funktionen bilden Einheit
(Attribute und Methoden: gekapselt).
Hauptaugenmerk: Kommunikation über Botschaften.



OOP

„Technically, there is no real difference between method calling and message passing.“

<https://www.panonit.com/blog/overview-message-passing-object-oriented-programming>

„Technically, there is no real difference between method calling and message passing.“

„But with the term “message passing” authors of this idea are trying to explain that focusing on the message can lead to creating better abstracting then just focusing on objects.“

<https://www.panonit.com/blog/overview-message-passing-object-oriented-programming>

„Technically, there is no real difference between method calling and message passing.“

„But with the term “message passing” authors of this idea are trying to explain that focusing on the message can lead to creating better abstracting then just focusing on objects.“

<https://www.panonit.com/blog/overview-message-passing-object-oriented-programming>



„Sending a message asking an object to apply a method is similar to a procedure call in ``traditional" programming languages.

<https://www.desy.de/gna/html/cc/Tutorial/node5.htm>

„Sending a message asking an object to apply a method is similar to a procedure call in ``traditional" programming languages. **However**, in object-orientation there is a view of autonomous objects which communicate with each other by exchanging messages. Objects react when they receive messages by applying methods on themselves. They also may deny the execution of a method, for example if the calling object is not allowed to execute the requested method. “

<https://www.desy.de/gna/html/cc/Tutorial/node5.htm>

Kommunikation: Beispiel Kontoüberweisung



```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag) {
        Kontostand += Betrag;
    }
    void Auszahlen(float Betrag) {
        Kontostand -= Betrag;
    }
    void AusgabeKontostand() {
        cout << "Der Kontostand beträgt: " <<
            Kontostand << endl;
    }
    void Ueberweisen(float Betrag,
                     konto* AufKonto) {
        Kontostand -= Betrag;
        AufKonto->Einzahlen(Betrag);
    }
private:
    float Kontostand = 0.0;
};
```

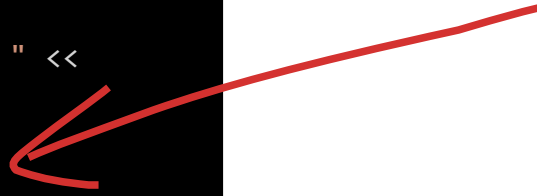


Kommunikation: Beispiel Kontoüberweisung



```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag) {
        Kontostand += Betrag;
    }
    void Auszahlen(float Betrag) {
        Kontostand -= Betrag;
    }
    void AusgabeKontostand() {
        cout << "Der Kontostand beträgt: " <<
            Kontostand << endl;
    }
    void Ueberweisen(float Betrag,
                     konto* AufKonto) {
        Kontostand -= Betrag;
        AufKonto->Einzahlen(Betrag);
    }
private:
    float Kontostand = 0.0;
};
```

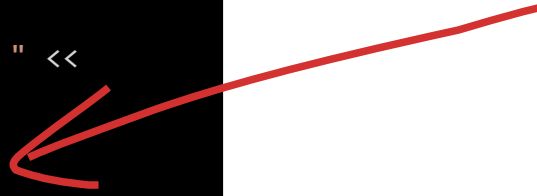


Kommunikation: Beispiel Kontoüberweisung



```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag) {
        Kontostand += Betrag;
    }
    void Auszahlen(float Betrag) {
        Kontostand -= Betrag;
    }
    void AusgabeKontostand() {
        cout << "Der Kontostand beträgt: " <<
            Kontostand << endl;
    }
    void Ueberweisen(float Betrag,
        konto* AufKonto) {
        Kontostand -= Betrag;
        AufKonto->Einzahlen(Betrag);
    }
private:
    float Kontostand = 0.0;
};
```

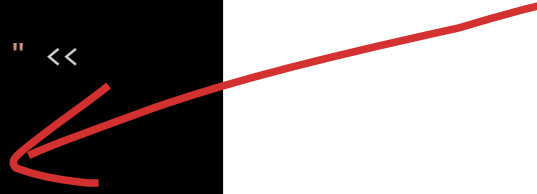


Kommunikation: Beispiel Kontoüberweisung



```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag) {
        Kontostand += Betrag;
    }
    void Auszahlen(float Betrag) {
        Kontostand -= Betrag;
    }
    void AusgabeKontostand() {
        cout << "Der Kontostand beträgt: " <<
            Kontostand << endl;
    }
    void Ueberweisen(float Betrag,
        konto* AufKonto) {
        Kontostand -= Betrag;
        AufKonto->Einzahlen(Betrag);
    }
private:
    float Kontostand = 0.0;
};
```



Kommunikation: Beispiel Kontoüberweisung



```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag) {
        Kontostand += Betrag;
    }
    void Auszahlen(float Betrag) {
        Kontostand -= Betrag;
    }
    void AusgabeKontostand() {
        cout << "Der Kontostand beträgt: " <<
            Kontostand << endl;
    }
    void Ueberweisen(float Betrag,
        konto* AufKonto) {
        Kontostand -= Betrag;
        AufKonto->Einzahlen(Betrag);
    }
private:
    float Kontostand = 0.0;
};
```

```
int main() {
    konto Konto1, Konto2;

    Konto1.Einzahlen(100.);
    Konto2.Einzahlen(200.);

    cout << "Kontostände vor Ueberweisung:" << endl;
    Konto1.AusgabeKontostand();
    Konto2.AusgabeKontostand();

    cout << "Ueberweisung 2->1:" << endl;
    Konto2.Ueberweisen(20.0, &Konto1);

    cout << "Kontostände nach Ueberweisung:" << endl;
    Konto1.AusgabeKontostand();
    Konto2.AusgabeKontostand();

    return 0;
}
```



Kommunikation: Beispiel Kontoüberweisung



```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag) {
        Kontostand += Betrag;
    }
    void Auszahlen(float Betrag) {
        Kontostand -= Betrag;
    }
    void AusgabeKontostand() {
        cout << "Der Kontostand beträgt: " <<
            Kontostand << endl;
    }
    void Ueberweisen(float Betrag,
                     konto* AufKonto) {
        Kontostand -= Betrag;
        AufKonto->Einzahlen(Betrag);
    }
private:
    float Kontostand = 0.0;
};
```

Was könnte hier schiefgehen?



- Kommerzielle Software enthält eine Vielzahl unterschiedlicher "Geschäftsobjekte".
- Um die Übersicht zu behalten und damit mehrere Teammitglieder gleichzeitig Änderungen machen können, wird der Source-Code modularisiert und auf mehrere .cpp-Dateien aufgeteilt. In den einzelnen .cpp-Dateien werden dann nur die Methoden einer einzelnen Klasse definiert.
- Damit andere Teile des Programms diese Definitionen nutzen können, müssen die Objekte (Eigenschaften und Methoden) jeweils als Deklaration eingebunden werden können.
- Diese Aufgabe erlauben sog. "Header-Dateien" (Kopf-Datei), die jeweils nur die Deklaration durchführen.
- Der Compiler übersetzt dann alle einzelnen .cpp-Dateien; der Linker fügt dann die Übersetzungen zu einem Programm zusammen.

Modulare Programmierung: Beispiel Kontoüberweisung



main.cpp

```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag) {
        Kontostand += Betrag;
    }
    void Auszahlen(float Betrag) {
        Kontostand -= Betrag;
    }
    void AusgabeKontostand() {
        cout << "Der Kontostand beträgt: " <<
            Kontostand << endl;
    }
    void Ueberweisen(float Betrag,
        konto* AufKonto) {
        Kontostand -= Betrag;
        AufKonto->Einzahlen(Betrag);
    }
private:
    float Kontostand = 0.0;
};
```

main.cpp

```
int main() {
    konto Konto1,Konto2;

    Konto1.Einzahlen(100.);
    Konto2.Einzahlen(200.);

    cout << "Kontostände vor Ueberweisung:" << endl;
    Konto1.AusgabeKontostand();
    Konto2.AusgabeKontostand();

    cout << "Ueberweisung 2->1:" << endl;
    Konto2.Ueberweisen(20.0, &Konto1);

    cout << "Kontostände nach Ueberweisung:" << endl;
    Konto1.AusgabeKontostand();
    Konto2.AusgabeKontostand();

    return 0;
}
```

← Beides in main.cpp

Modulare Programmierung: Beispiel Kontoüberweisung



main.cpp

```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag);
    void Auszahlen(float Betrag);
    void AusgabeKontostand();
    void Ueberweisen(float Betrag,
                     konto* AufKonto);
private:
    float Kontostand = 0.0;
};

void konto::Einzahlen(float Betrag)
{Kontostand += Betrag;}
void konto::Auszahlen(float Betrag)
{Kontostand -= Betrag;}
void konto::AusgabeKontostand() {
    cout << "Der Kontostand beträgt: " <<
        Kontostand << endl;}
void konto::Ueberweisen(float Betrag,
                        konto* AufKonto) {
    Kontostand -= Betrag;
    AufKonto->Einzahlen(Betrag);}
```

main.cpp

```
int main() {
    konto Konto1,Konto2;

    Konto1.Einzahlen(100.);
    Konto2.Einzahlen(200.);

    cout << "Kontostände vor Ueberweisung:" << endl;
    Konto1.AusgabeKontostand();
    Konto2.AusgabeKontostand();

    cout << "Ueberweisung 2->1:" << endl;
    Konto2.Ueberweisen(20.0, &Konto1);

    cout << "Kontostände nach Ueberweisung:" << endl;
    Konto1.AusgabeKontostand();
    Konto2.AusgabeKontostand();

    return 0;
}
```

BTW: Noch eine weitere Möglichkeit die
Klassendefinition zu strukturieren /
modularisieren



Modulare Programmierung: Beispiel Kontoüberweisung



konto.h

```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag);
    void Auszahlen(float Betrag);
    void AusgabeKontostand();
    void Ueberweisen(float Betrag,
                     konto* AufKonto);
private:
    float Kontostand = 0.0;
};

void konto::Einzahlen(float Betrag)
{Kontostand += Betrag;}
void konto::Auszahlen(float Betrag)
{Kontostand -= Betrag;}
void konto::AusgabeKontostand() {
    cout << "Der Kontostand beträgt: " <<
        Kontostand << endl;}
void konto::Ueberweisen(float Betrag,
                        konto* AufKonto) {
    Kontostand -= Betrag;
    AufKonto->Einzahlen(Betrag);}};
```

main.cpp

```
#include "konto.h"

int main() {
    konto Konto1,Konto2;

    Konto1.Einzahlen(100.);
    Konto2.Einzahlen(200.);

    [...]

    return 0;
}
```

Klassendefinition in eigener Datei....



Modulare Programmierung: Beispiel Kontoüberweisung



konto.h



main.cpp

```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag);
    void Auszahlen(float Betrag);
    void AusgabeKontostand();
    void Ueberweisen(float Betrag,
                     konto* AufKonto);
private:
    float Kontostand = 0.0;
};

void konto::Einzahlen(float Betrag)
{Kontostand += Betrag;}
void konto::Auszahlen(float Betrag)
{Kontostand -= Betrag;}
void konto::AusgabeKontostand() {
    cout << "Der Kontostand beträgt: " <<
        Kontostand << endl;}
void konto::Ueberweisen(float Betrag,
                        konto* AufKonto) {
    Kontostand -= Betrag;
    AufKonto->Einzahlen(Betrag);};
```

```
#include "konto.h"

int main() {
    konto Konto1,Konto2;

    Konto1.Einzahlen(100.);
    Konto2.Einzahlen(200.);

    [...]

    return 0;
}
```

Klassendefinition in eigener Datei....



Modulare Programmierung: Beispiel Kontoüberweisung



konto.h

```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag);
    void Auszahlen(float Betrag);
    void AusgabeKontostand();
    void Ueberweisen(float Betrag,
                     konto* AufKonto);
private:
    float Kontostand = 0.0;
};
```

```
#include "konto.h"
void konto::Einzahlen(float Betrag)
{Kontostand += Betrag;}
void konto::Auszahlen(float Betrag)
{Kontostand -= Betrag;}
void konto::AusgabeKontostand() {
    cout << "Der Kontostand beträgt: " <<
        Kontostand << endl;}
void konto::Ueberweisen(float Betrag,
                        konto* AufKonto) {
    Kontostand -= Betrag;
    AufKonto->Einzahlen(Betrag);}
```

konto.cpp

main.cpp

```
#include "konto.h"

int main() {
    konto Konto1,Konto2;

    Konto1.Einzahlen(100.);
    Konto2.Einzahlen(200.);

    [...]

    return 0;
}
```

Header-Dateien" (Kopf-Datei),
die jeweils nur die Deklaration durchführen.

CPP-Datei für die Implementation der Methoden.



Modulare Programmierung: Beispiel Kontoüberweisung



konto.h

```
#include <iostream>
using namespace std;

class konto {
public:
    konto(){}
    ~konto(){}
    void Einzahlen(float Betrag);
    void Auszahlen(float Betrag);
    void AusgabeKontostand();
    void Ueberweisen(float Betrag,
                     konto* AufKonto);
private:
    float Kontostand = 0.0;
};
```

main.cpp

```
#include "konto.h"

int main() {
    konto Konto1, Konto2;

    Konto1.Einzahlen(100.);
    Konto2.Einzahlen(200.);

    [...]

    return 0;
}
```

```
#include "konto.h"
void konto::Einzahlen(float Betrag)
{Kontostand += Betrag;}
void konto::Auszahlen(float Betrag)
{Kontostand -= Betrag;}
void konto::AusgabeKontostand() {
    cout << "Der Kontostand beträgt: " <<
         Kontostand << endl;}
void konto::Ueberweisen(float Betrag,
                        konto* AufKonto) {
    Kontostand -= Betrag;
    AufKonto->Einzahlen(Betrag);}
```

konto.cpp

Header-Datei (Kopf-Datei)
für die Deklaration

Quell-Datei für die Implementation der Methoden.



Bei der Vererbung wird eine neue Klasse von einer bestehenden Klasse abgeleitet.
Die abgeleitete Klasse **erbt** dadurch die Elemente ihrer Basisklasse.

- Die **Basisklasse** gibt ihre Elemente weiter (vererbt diese).
Alternativ wird von Superklasse gesprochen
- Die **abgeleitete Klasse** nimmt die Elemente (erbt diese).
Alternative wird von Subklasse gesprochen.

Bei der Vererbung wird eine neue Klasse von einer bestehenden Klasse abgeleitet.
Die abgeleitete Klasse **erbt** dadurch die Elemente ihrer Basisklasse.

- Die **Basisklasse** gibt ihre Elemente weiter (vererbt diese).
Alternativ wird von Superklasse gesprochen
- Die **abgeleitete Klasse** nimmt die Elemente (erbt diese).
Alternative wird von Subklasse gesprochen.

Vererbung lässt sich nicht in jedem Fall anwenden, allerdings wenn sie angewendet werden kann
„besticht [sie] durch [ihre] **Leistungsfähigkeit und Eleganz**“ D. Lois, C++

Vererbung



```
class Basis {  
public:  
    void verdoppeln() {  
        Wert *= 2;  
    }  
  
    void ausgabe() {  
        cout << Wert << endl;  
    }  
  
    int Wert = 10;  
  
};
```

```
int main()  
{  
    cout << "Demo Programm Vererbung!\n";  
  
    Basis test;  
    test.ausgabe();  
  
    return 0;  
}
```



Vererbung



```
class Basis {
public:
    void verdoppeln() {
        Wert *= 2;
    }

    void ausgabe() {
        cout << Wert << endl;
    }

    int Wert = 10;
};

class Ableitung : public Basis {
public:
    void halbieren() {
        Wert /= 2;
    }
};
```

```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    return 0;
}
```

Die abgeleitete Klasse („Ableitung“) erbt alle Elemente der Basisklasse („Basis“) und übernimmt die Zugriffsrechte unverändert (public).



Vererbung



```
class Basis {
public:
    void verdoppeln() {
        Wert *= 2;
    }

    void ausgabe() {
        cout << Wert << endl;
    }

    int Wert = 10;
};

class Ableitung: public Basis {
public:
    void halbieren() {
        Wert /= 2;
    }
};
```

```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    return 0;
}
```

Die **abgeleitete Klasse** („Ableitung“) erbt alle Elemente der Basisklasse („Basis“) und übernimmt die Zugriffsrechte unverändert (public).



Vererbung



```
class Basis {
public:
    void verdoppeln() {
        Wert *= 2;
    }

    void ausgabe() {
        cout << Wert << endl;
    }

    int Wert = 10;
};

class Ableitung : public Basis {
public:
    void halbieren() {
        Wert /= 2;
    }
};
```

```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    return 0;
}
```

Die abgeleitete Klasse („Ableitung“) erbt alle Elemente der **Basisklasse** („Basis“) und übernimmt die Zugriffsrechte unverändert (public).



Vererbung



```
class Basis {
public:
    void verdoppeln() {
        Wert *= 2;
    }

    void ausgabe() {
        cout << Wert << endl;
    }

    int Wert = 10;
};

class Ableitung : public Basis {
public:
    void halbieren() {
        Wert /= 2;
    }
};
```



```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    return 0;
}
```

Die abgeleitete Klasse („Ableitung“) erbt alle Elemente der Basisklasse („Basis“) und übernimmt die **Zugriffsrechte** unverändert (public).



Vererbung



```
class Basis {
public:
    void verdoppel() {
        Wert *= 2;
    }

    void ausgabe() {
        cout << Wert << endl;
    }

    int Wert = 10;
};

class Ableitung : public Basis {
public:
    void halbieren() {
        Wert /= 2;
    }
};
```

```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    Ableitung test2;
    test2.ausgabe();
    test2.verdoppel();
    test2.halbieren();
    test2.ausgabe();

    return 0;
}
```

Die abgeleitete Klasse („Ableitung“) erbt alle Elemente der Basisklasse („Basis“) und übernimmt die Zugriffsrechte unverändert (public).



Vererbung



```
class Basis {
public:
    void verdoppel() {
        Wert *= 2;
    }

    void ausgabe() {
        cout << Wert << endl;
    }

    int Wert = 10;
};

class Ableitung : private Basis {
public:
    void halbieren() {
        Wert /= 2;
    }
};
```



```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    Ableitung test2;
    test2.ausgabe();
    test2.verdoppel();
    test2.halbieren();
    test2.ausgabe();

    return 0;
}
```

Die abgeleitete Klasse („Ableitung“) erbt alle Elemente der Basisklasse („Basis“) und **verändert die Zugriffsrechte auf private (private).**



Vererbung



```
class Basis {
public:
    void verdoppeln() {
        Wert *= 2;
    }

    void ausgabe() {
        cout << Wert << endl;
    }

    int Wert = 10;

};

class Ableitung : private Basis {
public:
    void halbieren() {
        Wert /= 2;
    }

};
```

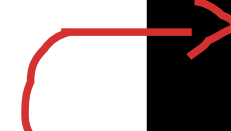


```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    Ableitung test2;
    test2.ausgabe();
    test2.verdoppeln();
    test2.halbieren();

    return 0;
}
```



✖ C2247 Kein Zugriff auf "Basis::ausgabe", da "Ableitung" ""private" verwendet, um von "Basis" zu erben

Die abgeleitete Klasse („Ableitung“) erbt alle Elemente der Basisklasse („Basis“) und **verändert die Zugriffsrechte auf private (private).**



Vererbung – private Members



```
class Basis {
public:
    void verdoppeln() {
        Wert *= 2;
    }

    void ausgabe() {
        cout << Wert << endl;
    }

private:
    int Wert = 10;
};

class Ableitung : public Basis {
public:
    void halbieren() {
        Wert /= 2;
    }

};
```

```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    Ableitung test2;
    test2.ausgabe();
    test2.verdoppeln();
    test2.halbieren();
    test2.ausgabe();

    return 0;
}
```

Private Memberfunktionen/-variablen werden vererbt..

Vererbung – private Members



```
class Basis {
public:
    void verdoppeln() {
        Wert *= 2;
    }

    void ausgabe() {
        cout << Wert << endl;
    }

private:
    int Wert = 10;
};

class Ableitung : public Basis {
public:
    void halbieren() {
        Wert /= 2;
    }
};
```



```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    Ableitung test2;
    test2.ausgabe();
    test2.verdoppeln();
    test2.halbieren();
    test2.ausgabe();

    return 0;
}
```

Private Memberfunktionen/-variablen werden vererbt, **aber auf diese kann nicht direkt zugegriffen werden!**



abc	E0265	Auf "Member "Basis::Wert" (deklariert in Zeile 23)" kann nicht zugegriffen werden.
✖	C2248	"Basis::Wert": Kein Zugriff auf private Member, dessen Deklaration in der Basis-Klasse erfolgte.

Vererbung – private Members



```
class Basis {
public:
    void verdoppel() Wert *= 2;
    void ausgabe() {
        cout << Wert << endl;
    }
    void set_Wert(int aufwas) {
        Wert = aufwas;
    }
    int get_Wert() {
        return Wert;
    }

private:
    int Wert = 10;
};

class Ableitung : public Basis {
public:
    void halbieren() {
        set_Wert( get_Wert() / 2);
    }

};
```

```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    Ableitung test2;
    test2.ausgabe();
    test2.verdoppel();
    test2.halbieren();
    test2.ausgabe();

    return 0;
}
```

Private Memberfunktionen/-variablen werden vererbt, aber auf diese kann nicht direkt zugegriffen werden!

- Verwenden von getter/setter Funktionen.



Vererbung – private Members



```
class Basis {
public:
    void verdoppel() Wert *= 2;
    void ausgabe() {
        cout << Wert << endl;
    }

protected:
    int Wert = 10;
};

class Ableitung : public Basis {
public:
    void halbieren() {
        Wert /= 2;
    }
};
```

```
int main()
{
    cout << "Demo Programm Vererbung!\n";

    Basis test;
    test.ausgabe();

    Ableitung test2;
    test2.ausgabe();
    test2.verdoppel();
    test2.halbieren();
    test2.ausgabe();

    return 0;
}
```

Private Memberfunktionen/-variablen werden vererbt, aber auf diese kann nicht direkt zugegriffen werden!

- Verwenden von getter/setter Funktionen.
- Verwenden von protected -> Klassenelemente so privatisiert, dass auf sie außer von der eigenen Klasse nur von der abgeleiteten Klasse zugegriffen werden kann.



Vererbung oder Komposition?



Vererbung

```
class X {  
    ...  
};  
  
class Y : public X {  
    ...  
};
```

Komposition

```
class X {  
    ...  
};  
  
class Y {  
    X var;  
};
```



Vererbung oder Komposition?



Vererbung

```
class X {  
    ...  
};  
  
class Y : public X {  
    ...  
};
```

Erlaubt es in der abgeleiteten Klasse die Funktionen der Basisklasse (außer privaten Elementen) zu nutzen.

Vorteile der Polymorphie (kommt später).

Komposition

```
class X {  
    ...  
};  
  
class Y {  
    X var;  
};
```



Vererbung oder Komposition?



Vererbung

```
class X {  
    ...  
};  
  
class Y : public X {  
    ...  
};
```

Erlaubt es in der abgeleiteten Klasse die Funktionen der Basisklasse (außer privaten Elementen) zu nutzen.

Vorteile der Polymorphie (kommt später).

Ist-ein Beziehung

Komposition

```
class X {  
    ...  
};  
  
class Y {  
    X var;  
};
```



Vererbung oder Komposition?



Vererbung

```
class X {  
    ...  
};  
  
class Y : public X {  
    ...  
};
```

Erlaubt es in der abgeleiteten Klasse die Funktionen der Basisklasse (außer privaten Elementen) zu nutzen.

Vorteile der Polymorphie (kommt später).

Ist-ein Beziehung

Komposition

```
class X {  
    ...  
};  
  
class Y {  
    X var;  
};
```

Durch Einbettung von Objekten deren Funktionalität (öffentlichen Schnittstellen) zu nutzen.

Im Vergleich zur Vererbung einfacher & sicherer.



Vererbung oder Komposition?



Vererbung

```
class X {  
    ...  
};  
  
class Y : public X {  
    ...  
};
```

Erlaubt es in der abgeleiteten Klasse die Funktionen der Basisklasse (außer privaten Elementen) zu nutzen.

Vorteile der Polymorphie (kommt später).

Ist-ein Beziehung

Komposition

```
class X {  
    ...  
};  
  
class Y {  
    X var;  
};
```

Durch Einbettung von Objekten deren Funktionalität (öffentlichen Schnittstellen) zu nutzen.

Im Vergleich zur Vererbung einfacher & sicherer.

Hat-ein Beziehung
(reines Nutzungsverhältnis)



Vererbung oder Komposition? Beispiel...



Vererbung

```
class Auto {  
    ...  
};  
  
class Sportwagen : public Auto {  
    ...  
};
```

Erlaubt es in der abgeleiteten Klasse die Funktionen der Basisklasse (außer privaten Elementen) zu nutzen.

Vorteile der Polymorphie (kommt später).

Ist-ein Beziehung

Komposition

```
class Motor {  
    ...  
};  
  
class Auto {  
    Motor var;  
};
```

Durch Einbettung von Objekten deren Funktionalität (öffentlichen Schnittstellen) zu nutzen.

Im Vergleich zur Vererbung einfacher & sicherer.

Hat-ein Beziehung
(reines Nutzungsverhältnis)



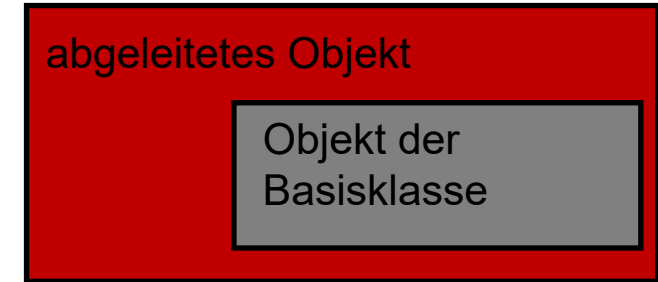
Vererbung: Basisklassenunterobjekte



Die von der Basisklasse geerbten Elemente bilden in der abgeleiteten Klasse eine eigenständige Untereinheit - ein Basisklassenobjekt.

In bestimmten Situationen muss das berücksichtigt werden:

- kein Zugriff auf private-Elemente der Basisklasse
- Basisklassen Memberfunktionen greifen auf Objekte der Basisklasse zurück, selbst wenn es gleichnamige im abgeleiteten Objekt gibt
- Instanzbildung - Unterobjekte müssen durch den Konstruktor der Basisklasse initialisiert werden
- Verdecken, Überschreiben, Überladen
- Mehrfachvererbung
- die Behandlung als Basisklassenobjekt
-> Polymorphie



Vererbung: Basisklassenunterobjekte



Die von der Basisklasse geerbten Elemente bilden in der abgeleiteten Klasse eine eigenständige Untereinheit - ein Basisklassenobjekt.

In bestimmten Situationen muss das berücksichtigt werden:

- kein Zugriff auf private-Elemente der Basisklasse
- **Basisklassen Memberfunktionen greifen auf Objekte der Basisklasse zurück, selbst wenn es gleichnamige im abgeleiteten Objekt gibt**
- Instanzbildung - Unterobjekte müssen durch den Konstruktor der Basisklasse initialisiert werden
- Verdecken, Überschreiben, Überladen
- Mehrfachvererbung
- die Behandlung als Basisklassenobjekt
-> Polymorphie

abgeleitetes Objekt

Objekt der
Basisklasse

```
class Basis {
protected:
    int Wert;
public:
    Basis(): Wert(1) {} // init mit 1
    void set_wert(int n) { Wert = n; }
    int get_wert() { return Wert; }
};

class Abgeleitet : public Basis {
    int Wert;
public:
    void zurueck_setzen() {Wert = 0;}
};

int main() {
    Abgeleitet obj;
    obj.set_wert(1000);
    obj.zurueck_setzen();
    cout << obj.get_wert() << endl;
    return 0;
}
```



Vererbung: Basisklassenunterobjekte



Die von der Basisklasse geerbten Elemente bilden in der abgeleiteten Klasse eine eigenständige Untereinheit - ein Basisklassenobjekt.

In bestimmten Situationen muss das berücksichtigt werden:

- kein Zugriff auf private-Elemente der Basisklasse
- **Basisklassen Memberfunktionen greifen auf Objekte der Basisklasse zurück, selbst wenn es gleichnamige im abgeleiteten Objekt gibt**
- Instanzbildung - Unterobjekte müssen durch den Konstruktor der Basisklasse initialisiert werden
- Verdecken, Überschreiben, Überladen
- Mehrfachvererbung
- die Behandlung als Basisklassenobjekt
-> Polymorphie

abgeleitetes Objekt

Objekt der
Basisklasse

```
class Basis {
protected:
    int Wert;
public:
    Basis(): Wert(1) {} // init mit 1
    void set_wert(int n) { Wert = n; }
    int get_wert() { return Wert; }
};

class Abgeleitet : public Basis {
public:
    void zurueck_setzen() {Wert = 0;}
};

int main() {
    Abgeleitet obj;
    obj.set_wert(1000);
    obj.zurueck_setzen();
    cout << obj.get_wert() << endl;
    return 0;
}
```

Vererbung: Basisklassenunterobjekte



Die von der Basisklasse geerbten Elemente bilden in der abgeleiteten Klasse eine eigenständige Untereinheit - ein Basisklassenobjekt.

In bestimmten Situationen muss das berücksichtigt werden:

- kein Zugriff auf private-Elemente der Basisklasse
- **Basisklassen Memberfunktionen greifen auf Objekte der Basisklasse zurück, selbst wenn es gleichnamige im abgeleiteten Objekt gibt**
- Instanzbildung - Unterobjekte müssen durch den Konstruktor der Basisklasse initialisiert werden
- Verdecken, Überschreiben, Überladen
- Mehrfachvererbung
- die Behandlung als Basisklassenobjekt
-> Polymorphie

abgeleitetes Objekt

Objekt der
Basisklasse

```
class Basis {
protected:
    int Wert;
public:
    Basis(): Wert(1) {} // init mit 1
    void set_wert(int n) { Wert = n; }
    int get_wert() { return Wert; }
};

class Abgeleitet : public Basis {
public:
    void zurueck_setzen() {Wert = 0;}
};

int main() {
    Abgeleitet obj;
    obj.set_wert(1000);
    obj.zurueck_setzen();
    cout << obj.get_wert() << endl;
    return 0;
}
```

Vererbung: Basisklassenunterobjekte



Die von der Basisklasse geerbten Elemente bilden in der abgeleiteten Klasse eine eigenständige Untereinheit - ein Basisklassenobjekt.

In bestimmten Situationen muss das berücksichtigt werden:

- kein Zugriff auf private-Elemente der Basisklasse
- Basisklassen Memberfunktionen greifen auf Objekte der Basisklasse zurück, selbst wenn es gleichnamige im abgeleiteten Objekt gibt
- **Instanzbildung - Unterobjekte müssen durch den Konstruktor der Basisklasse initialisiert werden**
- Verdecken, Überschreiben, Überladen
- Mehrfachvererbung
- die Behandlung als Basisklassenobjekt
-> Polymorphie

abgeleitetes Objekt

Objekt der
Basisklasse

```
class Basis {
protected:
    int Wert;
public:
    Basis(): Wert(1) {} // init mit =1
    void set_wert(int n) { Wert = n; }
    int get_wert() { return Wert; }
};

class Abgeleitet : public Basis {
public:
    Abgeleitet() {Wert += 10;}
};

int main() {
    Abgeleitet obj;
    cout << obj.get_wert() << endl;
    return 0;
}
```

Microsoft Visual Studio-Debu

11

K
A

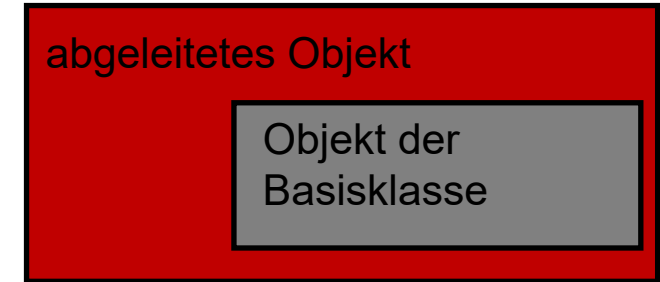
Vererbung: Basisklassenunterobjekte



Die von der Basisklasse geerbten Elemente bilden in der abgeleiteten Klasse eine eigenständige Untereinheit - ein Basisklassenobjekt.

In bestimmten Situationen muss das berücksichtigt werden:

- kein Zugriff auf private-Elemente der Basisklasse
- Basisklassen Memberfunktionen greifen auf Objekte der Basisklasse zurück, selbst wenn es gleichnamige im abgeleiteten Objekt gibt
- Instanzbildung - Unterobjekte müssen durch den Konstruktor der Basisklasse initialisiert werden
- **Verdecken, Überschreiben, Überladen**
- Mehrfachvererbung
- die Behandlung als Basisklassenobjekt
-> Polymorphie



Verdecken:

Membervariablen/-funktionen mit gleichem Namen wie ein geerbtes Element, verdecken diese. Auf das geerbte Element kann explizit zugegriffen werden: `Basis::Name`

Überladen:

Geerbte Memberfunktionen können nicht überladen werden, ggf. werden sie verdeckt.

Überschreiben:

Geerbte Memberfunktionen können überschrieben werden -> Polymorphie



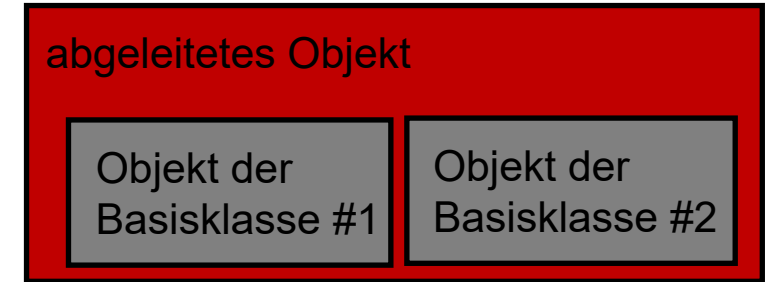
Vererbung: Basisklassenunterobjekte



Die von der Basisklasse geerbten Elemente bilden in der abgeleiteten Klasse eine eigenständige Untereinheit - ein Basisklassenobjekt.

In bestimmten Situationen muss das berücksichtigt werden:

- kein Zugriff auf private-Elemente der Basisklasse
- Basisklassen Memberfunktionen greifen auf Objekte der Basisklasse zurück, selbst wenn es gleichnamige im abgeleiteten Objekt gibt
- Instanzbildung - Unterobjekte müssen durch den Konstruktor der Basisklasse initialisiert werden
- Verdecken, Überschreiben, Überladen
- **Mehrfachvererbung**
- die Behandlung als Basisklassenobjekt
-> Polymorphie



```
class Abgeleitet : public Basis, public Basis2
```

„Die Mehrfachvererbung ist ein schwer zu handhabendes Konzept. das bei unnötigem Gebrauch meist mehr Probleme aufwirft, als es Hilfe bringt“ D. Lois C++



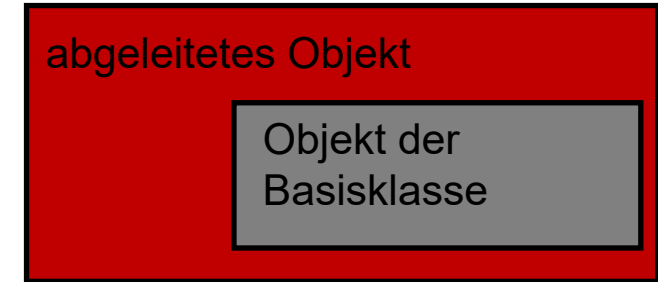
Vererbung: Basisklassenunterobjekte



Die von der Basisklasse geerbten Elemente bilden in der abgeleiteten Klasse eine eigenständige Untereinheit - ein Basisklassenobjekt.

In bestimmten Situationen muss das berücksichtigt werden:

- kein Zugriff auf private-Elemente der Basisklasse
- Basisklassen Memberfunktionen greifen auf Objekte der Basisklasse zurück, selbst wenn es gleichnamige im abgeleiteten Objekt gibt
- Instanzbildung - Unterobjekte müssen durch den Konstruktor der Basisklasse initialisiert werden
- Verdecken, Überschreiben, Überladen
- Mehrfachvererbung
- **die Behandlung als Basisklassenobjekt**
-> **Polymorphie**



Polymorphie



Polymorphie heißt auf Deutsch Vielgestaltigkeit.

„Polymorphie bezeichnet in der Chemie die Vielgestaltigkeit von Kristallen und in der Biologie die Vielgestaltigkeit von Tierstaaten wie Ameisen bezüglich ihrer Aufgabenverteilung.

Bei einer Programmiersprache: Statt Polymorphie könnte man auch von der »Selbstständigkeit des Objekts« sprechen, aber dafür gibt es keinen so schönen griechischen Ausdruck.“^[1]

„Objekte unterschiedlicher Typen [zeigen] gleiche Verhaltensweisen in allerdings eigener Ausprägung“^[2]

„In der OOP wird mit Polymorphismus die Fähigkeit einer Variable gemeint, zur Laufzeit eines Programms auf verschiedene Objekte (in C++ abgeleiteten Klassen zugeordnet) zu verweisen. Erst zur Laufzeit wird die zu dem jeweiligen Objekt passende Realisierung einer Operation ermittelt. Man spricht dann von dynamischen oder späten Binden der auszuführenden Anweisungen an einen Funktionsaufruf. Die zur Laufzeit gewählte Methode heißt virtuelle Funktion.“^[3]

[1] <http://www.willemer.de/informatik/cpp/cppvirt.htm>

[2] D. Lois, C++

[3] U. Breymann, Der C++ Programmierer



Polymorphie



```
#include <iostream>
using namespace std;

class Basis {
public:
    void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: ???
    objAK.Ausgabe();       // Ausgabe: ???
    return 0;
}
```



Polymorphie



```
#include <iostream>
using namespace std;

class Basis {
public:
    void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: Basisklasse
    objAK.Ausgabe();       // Ausgabe: abgeleitete Klasse
    return 0;
}
```



Polymorphie



```
#include <iostream>
using namespace std;

class Basis {
public:
    void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: Basisklasse
    objAK.Ausgabe();       // Ausgabe: abgeleitete Klasse
}
```

```
Basis* ptr = &objBasis;
ptr->Ausgabe();    // Ausgabe: ???

Abgeleitet* ptrAK = &objAK;
ptrAK->Ausgabe();    // Ausgabe: ???

return 0;
}
```



Polymorphie



```
#include <iostream>
using namespace std;

class Basis {
public:
    void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: Basisklasse
    objAK.Ausgabe();       // Ausgabe: abgeleitete Klasse
}
```

```
Basis* ptr = &objBasis;
ptr->Ausgabe();    // Ausgabe: Basisklasse

Abgeleitet* ptrAK = &objAK;
ptrAK->Ausgabe();    // Ausgabe: abgeleitete Klasse

return 0;
}
```



Polymorphie – Frühe Bindung



```
#include <iostream>
using namespace std;

class Basis {
public:
    void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: Basisklasse
    objAK.Ausgabe();       // Ausgabe: abgeleitete Klasse
}
```

```
Basis* ptr = &objBasis;
ptr->Ausgabe();           // Ausgabe: Basisklasse

Abgeleitet* ptrAK = &objAK;
ptrAK->Ausgabe();         // Ausgabe: abgeleitete Klasse

ptr = ptrAK;
ptr->Ausgabe();           // Ausgabe: ???

return 0;
}
```



Polymorphie – Frühe Bindung



```
#include <iostream>
using namespace std;

class Basis {
public:
    void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: Basisklasse
    objAK.Ausgabe();       // Ausgabe: abgeleitete Klasse
}
```

```
Basis* ptr = &objBasis;
ptr->Ausgabe();    // Ausgabe: Basisklasse

Abgeleitet* ptrAK = &objAK;
ptrAK->Ausgabe();    // Ausgabe: abgeleitete Klasse

ptr = ptrAK;
ptr->Ausgabe();    // Ausgabe: ???

return 0;
}
```

Bei der **frühen Bindung**:
Aufruf der Memberfunktion richtet sich
standardmäßig nach dem Typ des Zeigers!



Polymorphie – Frühe Bindung



```
#include <iostream>
using namespace std;

class Basis {
public:
    void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: Basisklasse
    objAK.Ausgabe();       // Ausgabe: abgeleitete Klasse
}
```

```
Basis* ptr = &objBasis;
ptr->Ausgabe();    // Ausgabe: Basisklasse

Abgeleitet* ptrAK = &objAK;
ptrAK->Ausgabe();    // Ausgabe: abgeleitete Klasse

ptr = ptrAK;
ptr->Ausgabe();    // Ausgabe: Basisklasse

return 0;
}
```

Bei der **frühen Bindung**:
Aufruf der Memberfunktion richtet sich
standardmäßig nach dem Typ des Zeigers!



Polymorphie – Späte Bindung (dynamisch)



```
#include <iostream>
using namespace std;

class Basis {
public:
    virtual void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    virtual void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: Basisklasse
    objAK.Ausgabe();       // Ausgabe: abgeleitete Klasse
}
```

```
Basis* ptr = &objBasis;
ptr->Ausgabe();    // Ausgabe: Basisklasse

Abgeleitet* ptrAK = &objAK;
ptrAK->Ausgabe();  // Ausgabe: abgeleitete Klasse

ptr = ptrAK;
ptr->Ausgabe();    // Ausgabe: ???

return 0;
}
```

Bei der **späten Bindung**:
Aufruf der Memberfunktion richtet sich nach
dem Typ des Objekts!



Polymorphie – Späte Bindung (dynamisch)



```
#include <iostream>
using namespace std;

class Basis {
public:
    virtual void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    virtual void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: Basisklasse
    objAK.Ausgabe();       // Ausgabe: abgeleitete Klasse
}
```

```
Basis* ptr = &objBasis;
ptr->Ausgabe();    // Ausgabe: Basisklasse

Abgeleitet* ptrAK = &objAK;
ptrAK->Ausgabe();    // Ausgabe: abgeleitete Klasse

ptr = ptrAK;
ptr->Ausgabe();    // Ausgabe: ???

return 0;
}
```

Bei der **späten Bindung**:
Aufruf der Memberfunktion richtet sich nach
dem Typ des Objekts!



Polymorphie – Späte Bindung (dynamisch)



```
#include <iostream>
using namespace std;

class Basis {
public:
    virtual void Ausgabe() {
        cout << "Basisklasse\n";
    }
};

class Abgeleitet : public Basis {
public:
    virtual void Ausgabe() {
        cout << "abgeleitete Klasse\n";
    }
};

int main()
{
    Basis objBasis;
    Abgeleitet objAK;
    objBasis.Ausgabe();    // Ausgabe: Basisklasse
    objAK.Ausgabe();       // Ausgabe: abgeleitete Klasse
}
```

```
Basis* ptr = &objBasis;
ptr->Ausgabe();    // Ausgabe: Basisklasse

Abgeleitet* ptrAK = &objAK;
ptrAK->Ausgabe();    // Ausgabe: abgeleitete Klasse

ptr = ptrAK;
ptr->Ausgabe();    // Ausgabe: abgeleitete Klasse

return 0;
}
```

Bei der **späten Bindung**:
Aufruf der Memberfunktion richtet sich nach
dem Typ des Objekts!





„Beim Studium der Polymorphie gewinnt mancher den Eindruck, dass sie nur für wenige, sehr konstruierte Beispiele einsetzbar wäre. Das ist keineswegs richtig.

Die meisten Klassenbibliotheken wären ohne Polymorphie gar nicht denkbar.

Insbesondere bei den grafischen Oberflächen stellen die Klassenbibliotheken Klassen für Applikationen, Fenster und Dialoge zur Verfügung. Um eigene Applikationen, Fenster oder Dialoge zu erstellen, leiten Sie von diesen Basisklassen Ihre eigenen Klassen ab und fügen die Elemente hinzu, die Ihr Programm von der Standardvorgabe unterscheiden.“^[1]

„Die Polymorphie ist der Grundstein für eine besondere Form der generischen OOP [...] die wichtigsten Einsatzgebiete hierfür sind Arrays von Basisklassentypen und Funktionen mit Basisklassenparametern“^[2]

[1] <http://www.willemer.de/informatik/cpp/cppvirt.htm>

[2] D. Lois, C++



Polymorphie – In der Praxis

„Die meisten Klassenbibliotheken wären ohne Polymorphie gar nicht denkbar.“^[1]

„Die Polymorphie ist der Grundstein für eine besondere Form der generischen OOP [...] die wichtigsten Einsatzgebiete hierfür sind Arrays von Basisklassentypen und Funktionen mit Basisklassenparametern“^[2]

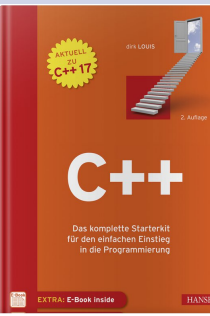
[1] <http://www.willemer.de/informatik/cpp/cppvirt.htm>

[2] D. Lois, C++

```
class Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Saeugetier." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};

class Gepard : public Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Gepard." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return true;
    }
};

class Elefant : public Saeugetier const
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Elefant." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};
```



Polymorphie – In der Praxis



```
class Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Saeugetier." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};

class Gepard : public Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Gepard." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return true;
    }
};

class Elefant : public Saeugetier const
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Elefant." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};
```

Listing 19.2 Einsatz generischer Arrays (Basisklassenarrays.cpp)

```
#include <iostream>
using namespace std;

class Saeugetier { wie oben }
class Gepard : public Saeugetier { wie oben }
class Elefant : public Saeugetier { wie oben }

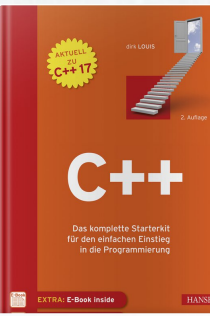
int main()
{
    cout << endl;

    Saeugetier *saeuger[5];

    saeuger[0] = new Gepard();
    saeuger[1] = new Elefant();
    saeuger[2] = new Elefant();
    saeuger[3] = new Elefant();
    saeuger[4] = new Gepard();

    cout << " Alle Tiere durchgehen: " << endl;

    for(int i = 0; i < 5; ++i)
        saeuger[i]->steckbrief();
}
```



Polymorphie – In der Praxis



```
class Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Saeugetier." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};

class Gepard : public Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Gepard." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return true;
    }
};

class Elefant : public Saeugetier const
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Elefant." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};
```

späte Bindung:
Methode des Obj.

Alle Tiere durchgehen:
Ich bin ein Gepard.
Ich bin ein Elefant.
Ich bin ein Elefant.
Ich bin ein Elefant.
Ich bin ein Gepard.

Listing 19.2 Einsatz generischer Arrays (Basisklassenarrays.cpp)

```
#include <iostream>
using namespace std;

class Saeugetier { wie oben }
class Gepard : public Saeugetier { wie oben }
class Elefant : public Saeugetier { wie oben }

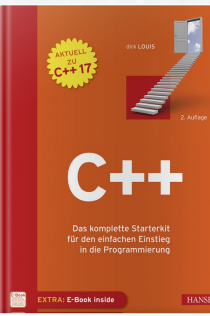
int main()
{
    cout << endl;

    Saeugetier *saeuger[5];

    saeuger[0] = new Gepard();
    saeuger[1] = new Elefant();
    saeuger[2] = new Elefant();
    saeuger[3] = new Elefant();
    saeuger[4] = new Gepard();

    cout << " Alle Tiere durchgehen: " << endl;

    for(int i = 0; i < 5; ++i)
        saeuger[i]->steckbrief();
}
```



Polymorphie – In der Praxis

```
class Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Saeugetier." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};

class Gepard : public Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Gepard." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return true;
    }
};

class Elefant : public Saeugetier const
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Elefant." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};
```

Listing 19.3 Einsatz generischer Memberfunktionen (Basisklassenparameter.cpp)

```
#include <iostream>
using namespace std;

class Saeugetier { wie oben }
class Gepard : public Saeugetier { wie oben }
class Elefant : public Saeugetier { wie oben }

void alarmieren(const Saeugetier& tier)
{
    if (tier.ist_raeuber() == true)
        cout << "\t Alarm! Ein Raeuber ist unterwegs." << endl;
    else
        cout << "\t Entwarnung! Nur ein harmloser Pflanzenfresser. "
            << endl;
}

int main()
{
    cout << endl;

    Saeugetier *saeuger[5];

    saeuger[0] = new Gepard();
    saeuger[1] = new Elefant();
    saeuger[2] = new Elefant();
    saeuger[3] = new Elefant();
    saeuger[4] = new Gepard();

    cout << " Alle Tiere durchgehen: " << endl;

    for(int i = 0; i < 5; ++i)
    {
        saeuger[i]->steckbrief();
        alarmieren(*saeuger[i]);
    }

    cout << endl;
    return 0;
}
```



Polymorphie – In der Praxis

```
class Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Saeugetier." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};
```

```
class Gepard : public Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Gepard." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return true;
    }
};
```

```
class Elefant : public Saeugetier
{
public:
    virtual void steckbrief() const
    {
        cout << " Ich bin ein Elefant." << endl;
    }
    virtual bool ist_raeuber() const
    {
        return false;
    }
};
```

Alle Tiere durchgehen:
Ich bin ein Gepard.
Alarm! Ein Raeuber ist unterwegs.
Ich bin ein Elefant.
Entwarnung! Nur ein harmloser Pflanzenfresser.
Ich bin ein Elefant.
Entwarnung! Nur ein harmloser Pflanzenfresser.
Ich bin ein Elefant.
Entwarnung! Nur ein harmloser Pflanzenfresser.
Ich bin ein Gepard.
Alarm! Ein Raeuber ist unterwegs.

Listing 19.3 Einsatz generischer Memberfunktionen (Basisklassenparameter.cpp)

```
#include <iostream>
using namespace std;
```

```
class Saeugetier { wie oben }
class Gepard : public Saeugetier { wie oben }
class Elefant : public Saeugetier { wie oben }
```

```
void alarmieren(const Saeugetier& tier)
```

Zugriff auf die Obj.

```
{
    if (tier.ist_raeuber() == true)
        cout << "\t Alarm! Ein Raeuber ist unterwegs." << endl;
    else
        cout << "\t Entwarnung! Nur ein harmloser Pflanzenfresser. "
            << endl;
}
```

```
int main()
```

```
{
    cout << endl;
```

```
Saeugetier *saeuger[5];
```

Array von Zeigern auf verschiedene Objekte mit gleicher Basisklasse

```
saeuger[0] = new Gepard();
saeuger[1] = new Elefant();
saeuger[2] = new Elefant();
saeuger[3] = new Elefant();
saeuger[4] = new Gepard();
```

```
cout << " Alle Tiere durchgehen: " << endl;
```

```
for(int i = 0; i < 5; ++i)
{
    saeuger[i]->steckbrief();
    alarmieren(*saeuger[i]);
}
```

```
cout << endl;
return 0;
}
```



+

K
A

„this“ = reserviertes Schlüsselwort!

Jedes Objekt/Instanz besitzt einen versteckten Zeiger „this“ auf das Objekt selbst.

Dieser versteckte Zeiger kann im erzeugten Assembler Quelltext gesehen werden, er wird automatisch vom Compiler angelegt.

Der this-Zeiger wird vor allem dazu verwendet, Verweise auf das aktuelle Objekt weiterzugeben oder die Verdeckung von Instanzvariablen durch gleichnamige Parameter oder lokale Variablen aufzulösen.

```
class Demo {
public:
    void Ausgabe() {
        cout << "Objekt: " << this << endl;
    }
    void funk(int n) {
        this->n = (n+1)*2;
    }
private:
    int n;
};

int main()
{
    Demo obj1, obj2;
    obj1.Ausgabe(); // Ausgabe: 0033F8F4
    obj2.Ausgabe(); // Ausgabe: 0033F8CC
    return 0;
}
```

„static“ = reserviertes Schlüsselwort!

Statische Membervariablen (Klassenvariablen)

- gibt es für jede Klasse nur einmal, d.h., sie werden nicht für die einzelnen Instanzen kopiert.
- eignen sich u.a. zum Speichern von Daten, die für alle Objekte der Klasse gleich sind.

Statische Memberfunktionen

- besitzen keinen this-Zeiger und können nur auf andere statische Elemente der Klasse zugreifen
- sind nicht an das Vorhandensein irgendwelcher Objekte der Klasse gebunden
- können daher z.B. als reine Funktionssammlungen dienen
- können zum setzen von Klassenvariablen verwendet werden.

```
class Demo {  
public:  
    static void setze_standardwert(int n) {  
        standardwert = n;  
    }  
private:  
    static int standardwert = 10;  
};  
  
int main()  
{  
    Demo obj1, obj2;  
    obj1.setze_standardwert(-1);  
    return 0;  
}
```

„static“ = reserviertes Schlüsselwort!

Statische Membervariablen (Klassenvariablen)

- gibt es für jede Klasse nur einmal, d.h., sie werden nicht für die einzelnen Instanzen kopiert.
- eignen sich u.a. zum Speichern von Daten, die für alle Objekte der Klasse gleich sind.

Statische Memberfunktionen

- besitzen keinen this-Zeiger und können nur auf andere statische Elemente der Klasse zugreifen
- sind nicht an das Vorhandensein irgendwelcher Objekte der Klasse gebunden
- können daher z.B. als reine Funktionssammlungen dienen
- können zum setzen von Klassenvariablen verwendet werden.

C++ Standard:

A member-declarator can contain a constant-initializer only if it declares a static member (9.4) of **const integral or const enumeration type**,

```
class Demo {  
public:  
    static void setze_standardwert(int n) {  
        standardwert = n;  
    }  
private:  
    static int standardwert = 10;  
    static constexpr float PI = 3.14159;  
};  
  
int main()  
{  
    Demo obj1, obj2;  
    obj1.setze_standardwert(-1);  
    return 0;  
}
```

- **Klassen** sind strukturierte Datentypen, die **Attribute**/Eigenschaften (Variablen) und **Methoden** (Funktionen) zusammenfassen.
- Die Instanzen eines solchen Datentyps werden **Objekte** (oder auch Instanzen) genannt.
- Die einzelnen Attribute und Methoden werden **Mitglied** (engl. **Member**) der Klasse genannt.
- Die Mitglieder einer Klassen können unterschiedliche **Sichtbarkeiten** haben:
 - öffentlich (engl. **public**) – Zugriff von außen (andere Klassen, Hauptprogramm) möglich
 - geschützt (engl. **protected**) – Zugriff nur innerhalb der eigenen Klassenhierarchie
 - privat (engl. **private**) – Zugriff nur aus der eigenen Klasse
- Klassen können von anderen Klassen **'erben'** (engl. "inherit, derive from"). Sie übernehmen dann die Mitglieder (Eigenschaften und Methoden) der beerbten Klasse (**Oberklasse**, engl. "super class") und werden dann zu einer abgeleiteten Klasse (**Unterklasse**). Dies formt eine **Klassenhierarchie**.
- Klassen können Methoden **'überladen'**. Sie belegen dann die Methode mit einem eigenen 'Verhalten', welches das Verhalten der Oberklasse ändert oder erweitert.
- Instanzen von Klassen einer Klassenhierarchie bieten **Polymorphie**: Der Aufruf einer Methode der Oberklasse aktiviert die entsprechende überladene Methode der jeweiligen Instanz (Überladung) oder erlaubt das dynamische oder späte Binden der auszuführenden Anweisungen an einen Funktionsaufruf.



Code aus der Vorlesung

```
#include <iostream>
#include "Auto.h" // <----
using namespace std;
```

```
int main() {

    Auto MeinAutoObj;
    Auto DeinAutoObj;
    MeinAutoObj.Anhupen(&DeinAutoObj);

    return 0;
}
```

main.cpp

```
class Auto {
public:
    Auto();
    void Starten();
    void Anhupen(Auto* wen);
    void AnGehuptwerden();
};
```

Auto.h

```
#include "Auto.h"
#include <iostream>
using namespace std;

Auto::Auto() {
    // Konstruktor
    cout << "ein Objekt wird erzeugt: " << this << endl;
}

void Auto::Anhupen(Auto* wen) {
    cout << this << ": HUUUPPP\n";
    wen->AnGehuptwerden();
}

void Auto::AnGehuptwerden() {
    cout << this << ": Troetoet\n";
}

void Auto::Starten() {
    cout << "BRUMBRUM\n";
}
```

Auto.cpp

Code aus der Vorlesung



```
#include <iostream>
#include "Auto.h" // <----
using namespace std;

class Motor {
public:
    int AnzahlZylinder = 8;
};

class Mercedes : public Auto {
public:
    bool hateinenStern = true;
    Motor BenzMotor;
};

class AMG : public Mercedes {
};

int main() {
    Mercedes MeinBenz;
    MeinBenz.Starten();
    if (MeinBenz.hateinenStern) cout << "Toll\n";
}
```



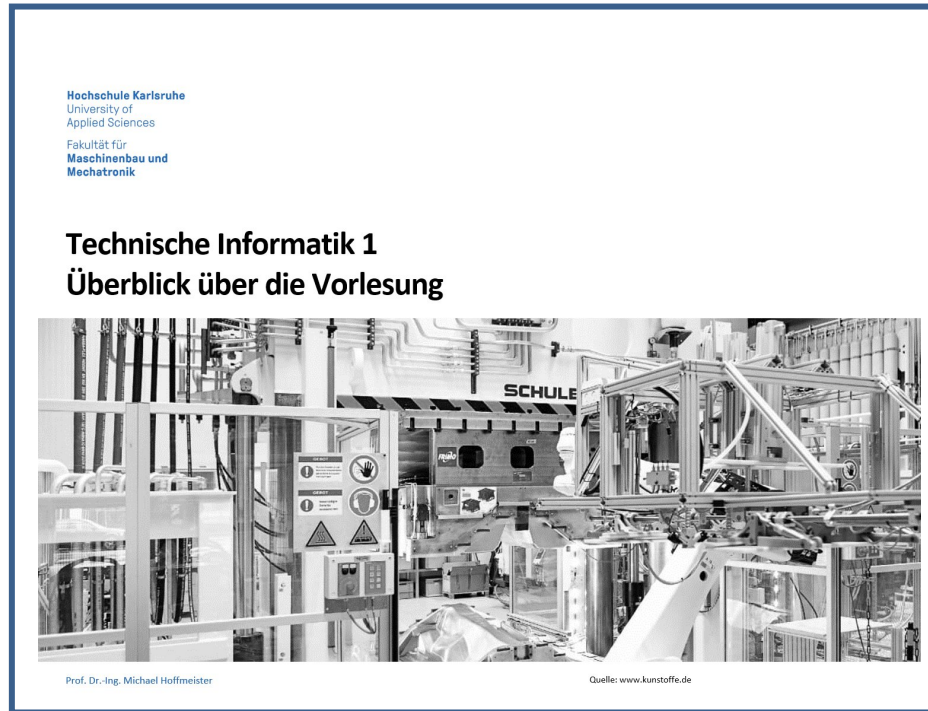
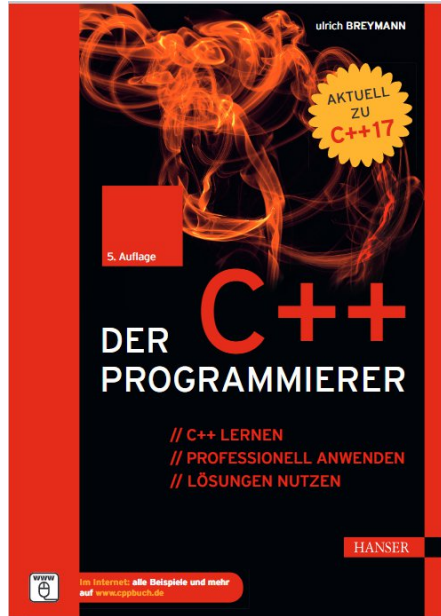
Code aus der Vorlesung

```
#include <iostream>
#include "Auto.h" // <----
using namespace std;

class Sinnvoll {
public:
    static constexpr float PI = 3.14159;
};

int main() {
    cout << Sinnvoll::PI << endl;
}
```





E2B240 Grundlagen der Informatik 2 - Christian Langen

Grundlagen der Informatik 2

Prof. Dr.-Ing. Christian Langen

Hochschule Karlsruhe –Technik und Wirtschaft
University of Applied Sciences
Fakultät für Elektro- und Informationstechnik (EIT)
Studiengang Elektrotechnik – Automatisierungstechnik (Bachelor) EAB

Sommersemester 2013

Prof. Dr.-Ing. Michael Hoffmeister

Prof. Dr.-Ing. Christian Langen